Execution Reference

Host Triage · Memory · Forensic Tools · Detection Queries

All runnable execution for a two-host investigation, organized by the five questions. Companion to the Analyst Methods Handbook and the Investigation Runbook.

Scenario assumed: one **Windows 10** workstation and one **Linux LAMP** server (Ubuntu likely). Tools: **Vol3** (Vol2 as fallback), **FTK Imager**, **FTK**, **Autopsy**. All live commands are read-only. Run as admin/root. Write output to external media or a path you note, never over evidence.

Section → question map. Every section answers one of the five questions. Tags like `(Q3)` in headings tell you which.

Q	Question	Windows	Linux	Memory	Disk
Q1	What is running that should not be?	§1.2	§2.2	§4, §5	§7D, §8B
Q2	What is talking that should not be?	§1.3	§2.3	§4, §5	logs
Q3	How does it survive a reboot?	§1.4	§2.4	svcscan, printkey	§7B paths
Q4	Who did it, and how did they get in?	§1.5, §1.7	§2.5, §2.7	getsids	logs
Q5	When did it start?	§1.6, §1.8	§2.6, §2.9	timeliner	§7B, §8B Timeline

Time rule: everything you record is UTC. Volatility prints UTC. PowerShell, Event Viewer, `ls`, Apache and syslog print the host's local time. Record each host's zone at orient (§1.1, §2.1) and convert in Hunt Notes or with the UTC forms below.

---

0. Drop-In Workflow

Order of operations. Memory first because every command you run changes it.

Phase	Goal	Time (3–4 hr window)	Section
0 Orient	What's provided: live access, images, tool versions, where to write output, each host's time zone	10–15 min	§0
1 Capture	Memory first (both hosts), then disk image if required	20–30 min	§3, §7A
2 Live triage	Q1 → Q5 shallow pass: procs → network → persistence → accounts → logs	30 min/host	§1, §2
3 Memory	Confirm/expand live findings, find hidden activity	40–45 min	§4–§6
4 Disk	Prove it on disk: files, execution artifacts, timestamps, logs	50–60 min	§7 (Imager, FTK), §8 (Autopsy)
5 Report	Findings with evidence + second source	20 min	§9

Breadth before depth: clear Q1–Q5 shallow on **both** hosts before going deep on any one artifact. Record the lead, continue, come back.

Orient checklist

```
[ ] Hostname / IP / OS build / system time zone recorded for each box
[ ] Am I admin/root?  (whoami /groups | findstr High  /  id)
[ ] Tool versions:  vol -h | head -1  ·  FTK Imager / FTK Help>About  ·  Autopsy Help>About
[ ] FTK: database service up, login works
[ ] Output location chosen and noted
[ ] Hunt Notes open, hosts added with their time zones (§9)
```

The pivot loop. Every lead runs the same circle until it closes (Q1 → Q2 → Q3 → Q4 → Q5):

```
suspicious PROCESS → PID / PPID / cmdline → NETWORK (who it talks to)
  → FILE on disk (path, hash, timestamps) → PERSISTENCE (how it survives)
  → ACCOUNT (who ran it, how they got in) → TIMELINE (first seen) → back to PROCESS
```

---

1. Win10 Live Triage (PowerShell, admin)

1.1 Context

```
hostname; whoami /all; (Get-CimInstance Win32_OperatingSystem) |
  Select Caption,Version,BuildNumber,LastBootUpTime
```

```
Get-Date -Format o; Get-TimeZone; (Get-Date).ToUniversalTime().ToString('u')
```

`Get-Date -Format o` shows the offset (e.g. `-10:00`). Put that zone on the host in Hunt Notes.

1.2 Processes (Q1)

```
Get-CimInstance Win32_Process |
  Select ProcessId,ParentProcessId,Name,@{n='CreatedUTC';e=
{$_.CreationDate.ToUniversalTime().ToString('u')}},ExecutablePath,CommandLine |
  Sort CreatedUTC |
  Format-Table -AutoSize -Wrap
```

Running from user-writable paths:

```
Get-CimInstance Win32_Process |
  ? { $_.ExecutablePath -match '\\(Temp|AppData|Users\\Public|ProgramData|Downloads)\\' } |
  Select ProcessId,ParentProcessId,Name,ExecutablePath,CommandLine
```

Unsigned running binaries:

```
Get-Process |
  ? Path |
  % { $s = Get-AuthenticodeSignature $_.Path; if ($s.Status -ne 'Valid') {
[pscustomobject]@{PID=$_.Id;Name=$_.Name;Path=$_.Path;Sig=$s.Status} } }
```

Parent of a PID:

```
$p=<PID>; Get-CimInstance Win32_Process -Filter "ProcessId=$p" |
  Select Name,ParentProcessId,CommandLine
```

Red flags: `winword/excel/outlook → cmd/powershell`, `w3wp/httpd → cmd`, `powershell -enc`, `rundll32` with no DLL args, `svchost` not parented by `services.exe`, `lsass` with a parent other than `wininit`.

1.3 Network (Q2)

```
Get-NetTCPConnection -State Established,Listen |
  Select LocalAddress,LocalPort,RemoteAddress,RemotePort,State,OwningProcess,@{n='Proc';e={(Get-
Process -Id $_.OwningProcess -EA 0).Name}} |
  Sort State,RemoteAddress |
  Format-Table -AutoSize
```

```
netstat -anob
```

```
Get-DnsClientCache |
  Select Entry,Data |
  Sort Entry
```

```
Get-Content C:\Windows\System32\drivers\etc\hosts |
  ? { $_ -notmatch '^\s*#' -and $_.Trim() }
```

1.4 Persistence (Q3)

Run keys:

```
'HKLM:\Software\Microsoft\Windows\CurrentVersion\Run','HKLM:\Software\Microsoft\Windows\CurrentVersion
\RunOnce','HKCU:\Software\Microsoft\Windows\CurrentVersion\Run','HKCU:\Software\Microsoft\Windows\Curr
entVersion\RunOnce','HKLM:\Software\WOW6432Node\Microsoft\Windows\CurrentVersion\Run' |
  % { if (Test-Path $_) { "== $_"; Get-ItemProperty $_ | Select * -Exclude PS* | Format-List } }
```

Services outside normal paths:

```
Get-CimInstance Win32_Service |
  ? { $_.PathName -notmatch 'System32|SysWOW64|Program Files' } |
  Select Name,State,StartMode,StartName,PathName
```

Scheduled tasks (non-Microsoft):

```
Get-ScheduledTask |
  ? TaskPath -notlike '\Microsoft\*' |
  Select TaskName,TaskPath,State,@{n='Run';e={($_.Actions | % { "$($_.Execute) $($_.Arguments)" }) -
join '; '}} |
  Format-Table -AutoSize -Wrap
```

WMI event subscriptions:

```
Get-CimInstance -Namespace root\subscription -ClassName __EventFilter; Get-CimInstance -Namespace root\subscription -ClassName CommandLineEventConsumer; Get-CimInstance -Namespace root\subscription -ClassName __FilterToConsumerBinding
```

Startup folders:

```
Get-ChildItem "C:\ProgramData\Microsoft\Windows\Start Menu\Programs\StartUp","C:\Users\*\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup" -Force -EA 0
```

Defender exclusions (attackers add these):

```
Get-MpPreference |
  Select ExclusionPath,ExclusionProcess,ExclusionExtension
```

Service DLLs outside System32 (svchost-hosted services hide here; the path filter above misses them):

```
Get-ChildItem HKLM:\SYSTEM\CurrentControlSet\Services -EA 0 |
  % { $d=(Get-ItemProperty "$($_.PSPath)\Parameters" -EA 0).ServiceDll; if ($d -and $d -notmatch '\\System32\\') { [pscustomobject]@{Service=$_.PSChildName;ServiceDll=$d} } }
```

Winlogon, IFEO debugger hijacks, BITS jobs:

```
Get-ItemProperty 'HKLM:\Software\Microsoft\Windows NT\CurrentVersion\Winlogon' | Select Shell,Userinit
Get-ChildItem 'HKLM:\Software\Microsoft\Windows NT\CurrentVersion\Image File Execution Options' -EA 0 |
  % { $g=(Get-ItemProperty $_.PSPath -EA 0).Debugger; if ($g) { "$($_.PSChildName) -> $g" } }
Get-BitsTransfer -AllUsers -EA 0 | Select JobId,DisplayName,TransferType,JobState,@{n='Files';e={$_.FileList.RemoteName -join '; '}}
```

Expected: `Shell = explorer.exe` , `Userinit = C:\Windows\system32\userinit.exe,` (trailing comma is normal). Anything appended is persistence.

1.5 Accounts (Q4)

```
Get-LocalUser |
  Select Name,Enabled,LastLogon,PasswordLastSet; Get-LocalGroupMember Administrators
```

```
quser 2>$null; Get-SmbShare; net session 2>$null
```

1.6 Files (Q1 / Q5)

Recently written files in common staging paths:

```
Get-ChildItem C:\Users\*\AppData,C:\Users\Public,C:\ProgramData,C:\Windows\Temp -Recurse -File -Force
-EA 0 |
  ? LastWriteTimeUtc -gt (Get-Date).ToUniversalTime().AddDays(-7) |
  Sort LastWriteTimeUtc -Desc |
  Select -First 50 LastWriteTimeUtc,Length,FullName
```

Prefetch (what ran recently; full method in §1.8):

```
Get-ChildItem C:\Windows\Prefetch -EA 0 |
  Sort LastWriteTimeUtc -Desc |
  Select -First 30 CreationTimeUtc,LastWriteTimeUtc,Name
```

PowerShell history (all users):

```
Get-ChildItem
C:\Users\*\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt -EA 0 |
  % { "== $($_.FullName)"; Get-Content $_ }
```

Hash a file:

```
Get-FileHash -Algorithm SHA256 '<path>'
```

1.7 Event Logs (Q4 / Q5)

`TimeCreated` is local time. The queries below add a `UTC` column; use it.

Log	ID	Meaning
Security	4624 / 4625	Logon success / fail (check LogonType: 2 interactive, 3 network, 10 RDP)
Security	4648	Logon with explicit credentials (runas, lateral movement)
Security	4672	Admin-equivalent logon
Security	4688	Process create (only if auditing is on)
Security	4720 / 4732	User created / added to group
Security	4697 / 4698 / 4702	Service installed / scheduled task created / task updated
Security	1102	Security log cleared
System	7045	Service installed

Log	ID	Meaning
System	104	Log cleared
PowerShell/Operational	4104	Script block
Defender/Operational	1116 / 1117	Detection / action
Sysmon/Operational	1, 3, 7, 8, 10, 11, 13, 22	Proc, net, image load, remote thread, process access (LSASS), file create, reg set, DNS

Logons with user, type and source IP:

```
Get-WinEvent -FilterHashtable @{LogName='Security';Id=4624} -MaxEvents 500 |
  % {
[pscustomobject]@{UTC=$_.TimeCreated.ToUniversalTime().ToString('u');User=$_.Properties[5].Value;Type=
$_.Properties[8].Value;IP=$_.Properties[18].Value;Proc=$_.Properties[17].Value} } |
  ? { $_.Type -in 3,10 } |
  Format-Table -AutoSize
```

Failed logons by source:

```
Get-WinEvent -FilterHashtable @{LogName='Security';Id=4625} -EA 0 |
  % { $_.Properties[19].Value } |
  Group |
  Sort Count -Desc |
  Select Count,Name
```

Service installs, task creation, log clears:

```
Get-WinEvent -FilterHashtable @{LogName='System';Id=7045,104} -EA 0 |
  Select @{n='UTC';e={$_.TimeCreated.ToUniversalTime().ToString('u')}},Id,Message |
  Format-List
Get-WinEvent -FilterHashtable @{LogName='Security';Id=4648,4697,4698,4720,4732,1102} -EA 0 |
  Select @{n='UTC';e={$_.TimeCreated.ToUniversalTime().ToString('u')}},Id,Message |
  Format-List
```

PowerShell script blocks with common bad terms:

```
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-PowerShell/Operational';Id=4104} -EA 0 |
  ? { $_.Message -match 'FromBase64String|IEX|Invoke-Expression|DownloadString|Net.WebClient|-enc' } |
  Select @{n='UTC';e={$_.TimeCreated.ToUniversalTime().ToString('u')}},@{n='Script';e=
{$_.Message.Substring(0,[Math]::Min(400,$_.Message.Length))}} |
  Format-List
```

Sysmon (if installed):

```
Get-WinEvent -LogName 'Microsoft-Windows-Sysmon/Operational' -MaxEvents 300 -EA 0 |
  ? Id -in 1,3,10,22 |
  Select @{n='UTC';e={$_.TimeCreated.ToUniversalTime().ToString('u')}},Id,Message |
  Format-List
```

1.8 Execution Artifacts, Native (Q5)

When did it first run? Anchor start time on execution artifacts **before** trusting file timestamps; file timestamps are the first thing an attacker alters. No Zimmerman tools needed. Every output below is already UTC, so it goes straight into Hunt Notes with zone `UTC`.

Artifact	Proves	Time it gives	Native read
Prefetch `C:\Windows\Prefetch\NAME-HASH.pf`	Program executed	`.pf` created ≈ first run · modified ≈ last run (last 8 run times are inside the file, needs a parser)	below
UserAssist (NTUSER)	GUI launch by that user	Last run + run count	below (ROT13 decode)
BAM/DAM (SYSTEM)	Executed, per user SID (Win10 1709+)	Last run	below
Amcache `C:\Windows\appcompat\Programs\Amcache.hve`	Present / executed, with SHA-1	Key last-write ≈ first seen	locked live: export with FTK Imager, view in Registry Viewer or Autopsy
Shimcache / AppCompatCache (SYSTEM)	File existed (execution not guaranteed on Win10)	File's own modified time	`vol windows.shimcachemem` (newer Vol3) or Vol2 `shimcache`
`$MFT` SI vs FN times	Timestomping	`$FILE_NAME` times are hard to fake	Autopsy → File Metadata tab (istat shows both), FTK Imager export

Prefetch, first and last run:

```
Get-ChildItem C:\Windows\Prefetch\*.pf -EA 0 |
  Select Name,@{n='FirstRunUTC';e={$_.CreationTimeUtc.ToString('u')}},@{n='LastRunUTC';e=
{$_.LastWriteTimeUtc.ToString('u')}} |
  Sort FirstRunUTC -Desc | Select -First 40
```

UserAssist, decoded (run per user; for another user's hive, export NTUSER.DAT and read offline):

```
Get-ChildItem 'HKCU:\Software\Microsoft\Windows\CurrentVersion\Explorer\UserAssist\*\Count' -EA 0 | %
{
  $k=$_; $k.GetValueNames() | % {
    $v=$_; $n=-join ($v.ToCharArray() | % { if ($_ -match '[A-Za-z]') { $b=if ([char]::IsUpper($_))
{65} else {97}; [char]((([int]$_ - $b + 13) % 26) + $b) } else { $_ } })
    $d=$k.GetValue($v); if ($d.Length -ge 68) { [pscustomobject]@{Program=$n;Runs=
[BitConverter]::ToInt32($d,4);LastRunUTC=
[DateTime]::FromFileTimeUtc([BitConverter]::ToInt64($d,60)).ToString('u')} } } } |
  ? Runs -gt 0 | Sort LastRunUTC -Desc
```

BAM, last run per user (Win10 1809+ path first, older path second):

```
'HKLM:\SYSTEM\CurrentControlSet\Services\bam\State\UserSettings','HKLM:\SYSTEM\CurrentControlSet\Servi
ces\bam\UserSettings' |
  ? { Test-Path $_ } | % { Get-ChildItem $_ } | % {
    $k=$_; $k.GetValueNames() | ? { $_ -like '\Device\*' } | % {
      [pscustomobject]@{SID=$k.PSChildName;Path=$_;LastRunUTC=
[DateTime]::FromFileTimeUtc([BitConverter]::ToInt64($k.GetValue($_),0)).ToString('u')} } } |
  Sort LastRunUTC -Desc
```

Resolve a SID: `(New-Object System.Security.Principal.SecurityIdentifier '<SID>').Translate([System.Security.Principal.NTAccount])`

Timestomp check (disk). In Autopsy, select the file → **File Metadata** tab. Compare `$STANDARD_INFORMATION` against `$FILE_NAME` times. SI earlier than FN, or SI with `.0000000` sub-seconds, points to timestomping. Use the FN time and the execution artifacts as the start-time source.

To timeline. One line per artifact, with that artifact's time, source = the artifact (`prefetch` , `userassist` , `bam` , `amcache` , `$MFT FN`), Q5. Two artifacts that agree on first run = a confirmed start time.

---

2. Linux Live Triage (root)

2.1 Context

```
hostname; uname -a; cat /etc/os-release |
  head -3; uptime; date; date -u; id
timedatectl 2>/dev/null | grep -i 'time zone' || cat /etc/timezone
```

Put that zone on the host in Hunt Notes. Apache logs carry their own offset; syslog/auth.log lines do not.

```
who -a; last -Faiw |
  head -30; lastb -Faiw 2>/dev/null |
  head -20
```

2.2 Processes (Q1)

```
ps -eo pid,ppid,user,lstart,etime,cmd --forest
```

Binaries deleted from disk or running from temp:

```
ls -l /proc/[0-9]*/exe 2>/dev/null |
  grep -E '\(deleted\)|/tmp/|/dev/shm/|/var/tmp/'
```

What is the web server spawning? (Shells under apache/www-data = webshell.)

```
ps -u www-data -f; pstree -apl $(pgrep -o apache2 || pgrep -o httpd)
```

Deep-dive one PID:

```
P=<PID>; ls -l /proc/$P/exe /proc/$P/cwd; tr '\0' ' ' < /proc/$P/cmdline; echo; tr '\0' '\n' <
/proc/$P/environ |
  head; ls -l /proc/$P/fd |
  head -30
```

2.3 Network (Q2)

```
ss -tulpn; ss -tanp state established
```

```
lsof -i -nP 2>/dev/null |
  grep -v LISTEN
```

```
cat /etc/hosts; cat /etc/resolv.conf
```

2.4 Persistence (Q3)

Cron, all users and system:

```
for u in $(cut -d: -f1 /etc/passwd); do crontab -l -u "$u" 2>/dev/null |
  grep -v '^#' |
  sed "s/^/[$u] /"; done; ls -la /etc/cron* /var/spool/cron/crontabs 2>/dev/null; cat /etc/crontab
```

systemd: **enabled** units (persistence that is stopped right now still survives reboot), then running, then timers:

```
systemctl list-unit-files --state=enabled --no-pager
systemctl list-units --type=service --state=running --no-pager; systemctl list-timers --all --no-pager
```

Recently modified unit files:

```
find /etc/systemd /lib/systemd /usr/lib/systemd /home/*/.config/systemd /root/.config/systemd -name
'*.service' -mtime -14 -ls 2>/dev/null
```

SSH keys, shell rc files, preload (rootkit tell):

```
find / -xdev -name authorized_keys -exec ls -la {} \; -exec cat {} \; 2>/dev/null
ls -la /etc/ld.so.preload 2>/dev/null &&
  cat /etc/ld.so.preload
ls -la /etc/rc.local /etc/profile.d/ 2>/dev/null; tail -5 /root/.bashrc /home/*/.bashrc
```

Other survivors: at jobs, SysV init, MOTD scripts, PAM hooks, file capabilities:

```
atq 2>/dev/null; ls -la /etc/init.d/ /etc/update-motd.d/ 2>/dev/null
grep -rnE 'pam_exec|pam_script' /etc/pam.d/ 2>/dev/null
getcap -r / 2>/dev/null
```

2.5 Accounts (Q4)

```
awk -F: '$3==0 {print "UID0:",$1}' /etc/passwd; awk -F: '$7 !~ /(nologin|false)$/ {print $1,$3,$6,$7}'
/etc/passwd
```

```
getent group sudo wheel; grep -vE '^\s*(#|$)' /etc/sudoers /etc/sudoers.d/* 2>/dev/null; ls -la --
time-style=full-iso /etc/passwd /etc/shadow /etc/group
```

Auth log (Ubuntu `auth.log`, RHEL `secure`):

```
grep -hE 'Accepted|Failed password|Invalid user|sudo:.*COMMAND|useradd|usermod' /var/log/auth.log*
/var/log/secure* 2>/dev/null |
  tail -100
```

```
grep -h 'Failed password' /var/log/auth.log* 2>/dev/null |
  grep -oE 'from [0-9.]+' |
  sort |
  uniq -c |
  sort -rn |
  head
```

No auth.log (journald-only systems). `--utc` prints UTC:

```
journalctl --utc -o short-iso _COMM=sshd --no-pager | tail -100
journalctl --utc -o short-iso _COMM=sudo --no-pager | tail -50
```

Shell history, all users:

```
for h in /root/.*_history /home/*/.*_history; do [ -f "$h" ] &&
  { echo "== $h"; cat "$h"; }; done
```

2.6 Files (Q1 / Q5)

```
ls -la /tmp /var/tmp /dev/shm; find /tmp /var/tmp /dev/shm -type f -exec file {} \; 2>/dev/null
```

Modified in the last 3 days (adjust):

```
find / -xdev -type f -mtime -3 ! -path '/proc/*' ! -path '/sys/*' ! -path '/run/*' ! -path
'/var/lib/*' 2>/dev/null |
  head -200
```

Between two times:

```
find / -xdev -type f -newermt '2026-10-19 00:00' ! -newermt '2026-10-21 00:00' 2>/dev/null |
  head -200
```

SUID binaries (compare against a clean box):

```
find / -xdev -perm -4000 -type f -exec ls -la {} \; 2>/dev/null
```

Package integrity (changed system binaries):

```
dpkg --verify 2>/dev/null |
  grep -v ' c ' |
  head -50    # Debian/Ubuntu
rpm -Va 2>/dev/null |
  grep -E '^..5' |
  head -50          # RHEL
```

Kernel modules / taint:

```
lsmod; cat /proc/sys/kernel/tainted; dmesg 2>/dev/null |
  grep -iE 'taint|module verification' |
  tail
```

2.7 Web Tier, Apache / PHP (Q1 / Q4)

Webroot location:

```
grep -RhiE '^\s*DocumentRoot' /etc/apache2 /etc/httpd 2>/dev/null
```

Recent PHP and webshell patterns:

```
find /var/www -type f -name '*.php*' -mtime -30 -ls 2>/dev/null
grep -RnliE 'eval\(|base64_decode|gzinflate|str_rot13|shell_exec|passthru|system\(|proc_open|assert\
(|\$_(GET|POST|REQUEST|COOKIE)\[' /var/www --include='*.php*' 2>/dev/null
```

Files in webroot not owned by the expected user:

```
find /var/www ! -user www-data ! -user root -ls 2>/dev/null |
  head
```

Access log (Ubuntu `/var/log/apache2/` , RHEL `/var/log/httpd/`):

```
L=/var/log/apache2/access.log
awk '{print $1}' $L |
  sort |
  uniq -c |
  sort -rn |
  head                     # top client IPs
awk '$6 ~ /POST/ {print $7}' $L |
  sort |
  uniq -c |
  sort -rn |
  head         # POST targets
grep -iE 'cmd=|exec=|c=|union.*select|\.\./|/etc/passwd|wget|curl|base64|%3Cscript' $L |
  tail -50
awk -F'"' '{print $6}' $L |
  sort |
  uniq -c |
  sort -rn |
  head               # user agents
awk '{print $9}' $L |
  sort |
  uniq -c |
  sort -rn                          # status codes
```

First and last hit for an attacker IP:

```
grep -h '<IP>' /var/log/apache2/access.log.1 /var/log/apache2/access.log 2>/dev/null |
  sed -n '1p;$p'
```

PHP errors often show the upload or exploit:

```
tail -100 /var/log/apache2/error.log
```

2.8 Database, MySQL / MariaDB (Q3 / Q4)

```
mysql -u root -p -e "SELECT user,host,plugin FROM mysql.user; SELECT * FROM mysql.func; SHOW VARIABLES
LIKE 'secure_file_priv'; SHOW VARIABLES LIKE 'general_log%';"
```

Any row in `mysql.func` is a user-defined function. A UDF loaded from the plugin dir is a known way to get OS command execution from the DB.

```
ls -la /var/log/mysql/; grep -iE 'into outfile|into dumpfile|load_file|grant all' /var/log/mysql/*.log
2>/dev/null |
  tail
```

2.9 File Times and Timeline (Q5)

`ls` / `find` print local time. Use these forms for UTC and birth time:

```
TZ=UTC stat -c '%n | birth %w | mod %y | change %z | access %x' <file>
TZ=UTC find /var/www /tmp /var/tmp /dev/shm -type f -newermt '2026-10-19 00:00' -printf '%TY-%Tm-%Td
%TT  %p\n' 2>/dev/null | sort
```

`birth -` means the kernel/coreutils can't report it. On ext4, read the inode's crtime (debugfs opens read-only by default):

```
debugfs -R "stat <$(stat -c %i <file>)>" "$(df --output=source <file> | tail -1)" 2>/dev/null | grep -
E 'crtime|mtime|ctime'
```

`ctime` newer than `mtime` means the mtime was set back (`touch -d`), which is the Linux timestomp tell.

Filesystem timeline from an image (Sleuth Kit, on the analysis box):

```
mmls image.dd                                  # find the partition offset (sectors)
fls -r -m / -o <OFFSET> image.dd > body.txt
mactime -b body.txt -d -z UTC 2026-10-19..2026-10-22 > fs-timeline.csv
```

Read `fs-timeline.csv` against `access.log` and `auth.log` around the first suspicious hit.

---

3. Memory Capture (Setup)

Win10 — FTK Imager (run from USB): `File → Capture Memory` → destination on external/output path → filename `HOST_mem.mem` → optional **Include pagefile** → Capture. Hash it after.

Alternative if winpmem is provided:

```
.\winpmem_mini_x64.exe E:\HOST_mem.raw
```

Linux — AVML (static binary, no build needed, preferred):

```
sudo ./avml /mnt/usb/HOST_mem.lime &&
  sha256sum /mnt/usb/HOST_mem.lime
```

Linux — LiME (module must match the running kernel):

```
sudo insmod lime-$(uname -r).ko "path=/mnt/usb/HOST_mem.lime format=lime"
```

Record the kernel. You need it for symbols:

```
uname -r; cat /proc/version
```

---

4. Volatility 3 — Windows

Invocation is `vol` (pip) or `python3 vol.py` (git). Add `-r csv > out.csv` to save, `-o <dir>` for dumps. **Volatility timestamps are UTC.**

```
vol -f mem.raw windows.info
```

Step	Plugin	Why
Process list	`windows.pslist` · `windows.pstree`	Baseline and parentage
Hidden / exited	`windows.psscan`	In psscan but not pslist = hidden or terminated
Command lines	`windows.cmdline` · `windows.cmdscan` · `windows.consoles` (newer Vol3)	Arguments, encoded PowerShell, typed console commands
Network	`windows.netscan`	Connections → PID
Injection	`windows.malfind`	RWX private memory with PE/shellcode
Modules	`windows.dlllist --pid N` · `windows.ldrmodules --pid N`	Odd paths; unlinked DLLs (False in lists)
Services	`windows.svcscan`	Odd binaries / names
Handles	`windows.handles --pid N`	Files, mutexes, registry keys
Persistence	`windows.registry.printkey --key "Software\Microsoft\Windows\CurrentVersion\Run"`	Run keys (needs hive offset for NTUSER)
Accounts	`windows.getsids --pid N` · `windows.hashdump`	Who ran it
Files	`windows.filescan`	Find file objects, then dump
Kernel / rootkit	`windows.ssdt` · `windows.callbacks` · `windows.driverscan` · `windows.modscan`	Hooks, unknown drivers
Execution history	`windows.shimcachemem` (newer Vol3)	Shimcache entries from memory (Q5)
Timeline	`timeliner.Timeliner`	All timestamped artifacts

Typical run:

```
vol -f mem.raw windows.pstree
vol -f mem.raw windows.psscan > psscan.txt; vol -f mem.raw windows.pslist > pslist.txt   # diff PIDs
vol -f mem.raw windows.cmdline
vol -f mem.raw windows.netscan
vol -f mem.raw windows.malfind
```

Pivot on a PID:

```
vol -f mem.raw windows.dlllist --pid <PID>
vol -f mem.raw windows.handles --pid <PID>
vol -f mem.raw -o dump/ windows.pslist --pid <PID> --dump
vol -f mem.raw -o dump/ windows.malfind --pid <PID> --dump
```

Find and dump a file:

```
vol -f mem.raw windows.filescan |
  grep -i '<name>'
vol -f mem.raw -o dump/ windows.dumpfiles --virtaddr <OFFSET>
```

Registry hives, then a key:

```
vol -f mem.raw windows.registry.hivelist
vol -f mem.raw windows.registry.printkey --offset <HIVE_OFFSET> --key
"Software\Microsoft\Windows\CurrentVersion\Run"
```

List plugin names in this build (they shift between versions):

```
vol -h |
  grep -E '^\s+windows\.'
```

---

5. Volatility 3 — Linux

Symbols are the blocker. Vol3 Linux plugins need an ISF JSON that matches the exact kernel.

1. Get the banner:

```
vol -f mem.lime banners.Banners
```

2. Check whether the symbols are already present:

```
vol isfinfo.IsfInfo 2>/dev/null |
  grep -i linux; ls volatility3/symbols/linux/ 2>/dev/null
```

3. If missing, build on a box with the matching kernel debug image (Ubuntu: `linux-image-$(uname -r)-dbgsym` from ddebs.ubuntu.com):

```
./dwarf2json linux --elf /usr/lib/debug/boot/vmlinux-<KVER> --system-map /boot/System.map-<KVER> >
ubuntu-<KVER>.json
cp ubuntu-<KVER>.json <volatility3>/volatility3/symbols/linux/
```

Step	Plugin	Why
Processes	`linux.pslist` · `linux.pstree` · `linux.psaux`	List, tree, full args
Shell history	`linux.bash`	Commands typed, even if history file was wiped
Network	`linux.sockstat` (older: `linux.netstat`)	Sockets → PID
Open files	`linux.lsof --pid N`	Files, sockets per PID
Injection	`linux.malfind`	RWX regions
Env	`linux.envars --pid N`	LD_PRELOAD, odd PATH
Maps	`linux.proc.Maps --pid N`	Mapped libs, anonymous exec regions
Rootkit	`linux.check_syscall` · `linux.check_modules` · `linux.lsmod` · `linux.hidden_modules` · `linux.tty_check` · `linux.check_afinfo`	Hooked syscalls, hidden LKMs

```
vol -f mem.lime linux.pstree
vol -f mem.lime linux.psaux
vol -f mem.lime linux.bash
vol -f mem.lime linux.sockstat
vol -f mem.lime linux.malfind
vol -f mem.lime linux.check_syscall; vol -f mem.lime linux.check_modules
```

Plugin names in this build:

```
vol -h |
  grep -E '^\s+linux\.'
```

---

6. Volatility 2 Fallback

Use when Vol3 errors on the image, symbols are missing and a Vol2 profile is available, or you want a plugin your Vol3 build lacks (`psxview` , `mftparser` , `shimcache` ; recent Vol3 releases added `cmdscan` / `consoles` . Check `vol -h`). Vol2 support stops around 2019–2020 Win10 builds. A newer build may not have a profile; stay in Vol3 then.

6.1 Windows

Profile:

```
vol.py -f mem.raw imageinfo           # slow; or:
vol.py -f mem.raw kdbgscan
vol.py --info |
  grep Win10             # pick closest Win10x64_<build>
```

Set the profile once:

```
export VOLATILITY_PROFILE=Win10x64_<BUILD>; export VOLATILITY_LOCATION=file://$(pwd)/mem.raw
```

Vol3	Vol2 equivalent
windows.pslist / pstree / psscan	`pslist` · `pstree` · `psscan` · **`psxview`** (cross-view hidden procs)
windows.cmdline	`cmdline` · **`cmdscan`** · **`consoles`** (typed commands + output)
windows.netscan	`netscan`
windows.malfind	`malfind -D out/`
windows.dlllist / ldrmodules	`dlllist -p N` · `ldrmodules -p N`
windows.handles	`handles -p N -t File,Mutant,Key`
windows.svcscan	`svcscan`
registry.printkey	`printkey -K "Software\Microsoft\Windows\CurrentVersion\Run"`
windows.hashdump	`hashdump`
windows.filescan / dumpfiles	`filescan` · `dumpfiles -Q <offset> -D out/`
pslist --dump	`procdump -p N -D out/`
ssdt / driverscan / modscan	`ssdt` · `driverirp` · `modscan` · `apihooks`
timeliner	`timeliner` · **`mftparser`** · **`shimcache`** · `userassist`

```
vol.py psxview; vol.py cmdscan; vol.py consoles; vol.py netscan; vol.py malfind -D out/
```

6.2 Linux

Needs a profile zip built against the target kernel:

```
cd volatility/tools/linux &&
  make &&
  zip Ubuntu_<KVER>.zip module.dwarf /boot/System.map-<KVER>
cp Ubuntu_<KVER>.zip ../../volatility/plugins/overlays/linux/
vol.py --info |
  grep Linux           # get profile name, e.g. LinuxUbuntu_<KVER>x64
```

```
vol.py --profile=<P> -f mem.lime linux_pstree
vol.py --profile=<P> -f mem.lime linux_psaux
vol.py --profile=<P> -f mem.lime linux_bash
vol.py --profile=<P> -f mem.lime linux_netstat
vol.py --profile=<P> -f mem.lime linux_lsof -p <PID>
vol.py --profile=<P> -f mem.lime linux_malfind
vol.py --profile=<P> -f mem.lime linux_check_syscall; vol.py --profile=<P> -f mem.lime
linux_hidden_modules
vol.py --profile=<P> -f mem.lime linux_enumerate_files |
  grep -E '/tmp|/dev/shm'
```

7. FTK Imager

FTK Imager is acquisition and preview. It is not an analysis suite. Use it to capture, browse, export and hash, then analyze in Autopsy, FTK, or Vol. Full **FTK** (Forensic Toolkit) is covered in §7C–§7D. Expect to use Imager to produce the image and FTK to analyze it.

7A. Setup (capture, load, verify)

Capture memory (live Win10): `File → Capture Memory` → set destination and filename → Include pagefile (optional) → Capture Memory.

Image a disk:

1. `File → Create Disk Image`
2. Source: **Physical Drive** (whole disk), **Logical Drive** (one volume), **Image File** (convert), or **Contents of a Folder**
3. Add destination → type **E01** (compressed, carries hashes) or **Raw (dd)**
4. Fill evidence info (case, examiner, notes)
5. Check **Verify images after they are created** → Start
6. Keep the generated `.txt` log. It holds the MD5/SHA1.

Grab locked registry/system files from a live box: `File → Obtain Protected Files` → destination → **Password recovery and all registry files**.

Load evidence to browse: `File → Add Evidence Item` → Physical Drive / Logical Drive / Image File / Contents of a Folder → select.

Mount an image read-only (so other tools can read it): `File → Image Mounting` → image file → Mount Type **Physical & Logical** → Mount Method **Block Device / Read Only** → Mount. It shows up as a drive letter.

Hash verify an existing image: right-click the evidence item → **Verify Drive/Image**.

7B. Hunt Navigation

Screen layout. Evidence Tree (top-left) → File List (top-right) → Viewer (bottom: Text / Hex / Auto). Properties pane on the left.

- Red **X** on an icon = deleted file (still recoverable if not overwritten)
- Click a column header in File List to sort. Sort **Date Modified** descending to see recent activity.
- Right-click → **Export Files** (copy out for hashing or other tools) or **Export File Hash List**

- `View → Hex` to inspect headers (`MZ` = PE executable, `7F 45 4C 46` = ELF)

Where to go — Windows image (`[root]` under the NTFS partition):

Path	What it tells you
`$MFT` (root)	Every file record. Export for MFTECmd/Autopsy
`Windows\System32\config\`	SAM, SYSTEM, SOFTWARE, SECURITY hives
`Users\<u>\NTUSER.DAT` · `AppData\Local\Microsoft\Windows\UsrClass.dat`	User hive: Run keys, RecentDocs, ShellBags
`Windows\System32\winevt\Logs\`	EVTX: Security, System, PowerShell/Operational, Sysmon
`Windows\Prefetch\`	Execution evidence (`NAME-HASH.pf`)
`Windows\System32\Tasks\`	Scheduled task XML
`ProgramData\Microsoft\Windows\Start Menu\Programs\StartUp\`	All-users startup
`Users\<u>\AppData\Roaming\Microsoft\Windows\Recent\`	LNK files: what was opened
`Users\<u>\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\`	PS history
`Users\<u>\Downloads` , `AppData\Local\Temp` , `Users\Public`	Staging locations
`$Recycle.Bin\<SID>\`	`$I` (metadata) and `$R` (content) pairs
`inetpub\logs\LogFiles\`	IIS logs, if present

Where to go — Linux image (ext4 partition):

Path	What it tells you
`/var/log/auth.log*` · `syslog*` · `apache2/` · `mysql/`	Logons, sudo, web hits, DB
`/var/www/`	Webroot: webshells, modified PHP
`/etc/passwd` · `shadow` · `group` · `sudoers.d/`	Accounts and privilege
`/etc/cron*` · `/var/spool/cron/crontabs/`	Cron persistence
`/etc/systemd/system/` · `/lib/systemd/system/`	Service persistence
`/root/` · `/home/*/` → `.bash_history` , `.ssh/authorized_keys` , `.bashrc`	User activity and backdoors
`/tmp/` · `/var/tmp/` · `/dev/shm/` (if captured)	Dropped tools
`/etc/ld.so.preload`	Userland rootkit

7C. FTK (Forensic Toolkit) — Setup

FTK is a case-based suite backed by a database. Processing (hashing, indexing, carving) runs up front, then you search and filter. Menu names shift between FTK 7 and 8; the flow below holds.

Before anything: confirm the database service is running and you can log in. FTK will not open a case without it. Credentials are usually set by whoever built the box. Ask, don't guess.

New case:

1. `File → New Case` (or **Case Manager → New**)
2. Case name `CASE_<host>`, description, case folder on the output drive
3. **Processing profile:** pick **Forensics** or a custom profile. Avoid **Field Mode**, which skips the indexing you'll search on.
4. Click **Customize** on the profile and set:

Turn on	Turn off (time sinks)
MD5 + SHA-1 hashes	Explicit Image Detection
Flag Bad Extensions	Optical Character Recognition
Expand Compound Files	Data Carving (run later if needed)
dtSearch Text Index	Thumbnails for videos
KFF (Known File Filter), if a KFF library is configured	Cerberus malware triage, unless you want it and time allows
Registry Reports	
Create Thumbnails for Graphics (optional)	

5. OK → **Add Evidence** dialog opens

Add evidence (`Evidence → Add/Remove` later):

- **Acquired Image of Drive** — E01 / raw from FTK Imager (most common)
- **All Images in Directory** — multiple images at once
- **Contents of a Folder** / **Individual File** — exported artifacts, memory-dump outputs
- **Physical Drive** / **Logical Drive** — attached media
- Set **time zone = the target's**. Wrong time zone corrupts every timestamp you report. Convert what you record to UTC (Hunt Notes, zone = that host).

Processing: watch the **Progress** window. You can start browsing as categories populate, but Index Search results are incomplete until indexing finishes.

Memory images: some FTK builds accept a memory dump as evidence and parse processes and sockets. Don't rely on it. Do memory in Vol3/Vol2 (§4–§6) and add dumped files back as **Individual File** evidence.

Add more processing later: `Evidence → Additional Analysis` (carving, more hashing, KFF) without rebuilding the case.

7D. FTK — Hunt Navigation

Tabs across the top. Work them in this order.

Tab	Use it for
Overview	Start here. Counts by **File Status** (Bad Extension, Deleted, Encrypted, **KFF Alert**, KFF Ignorable) and **File Category** (Executable, Documents, Archives)
Explore	File-system tree, same as FTK Imager. Use for the path tables in §7B
Index Search	Fast keyword search over the dtSearch index (IP, filename, `eval(` , `base64_decode`)
Live Search	Hex/text pattern search (slower, no index needed). Use for hex signatures
Bookmarks	Your marked evidence. Feeds the report
Graphics · **Email**	Only if relevant

First moves:

1. **Overview → KFF Alert.** Known-bad hash hits. Fastest win if a KFF library is loaded.
2. **Overview → Bad Extension.** Renamed executables (`.jpg` that's a PE).
3. **Overview → File Category → Executable.** Sort by Created/Modified. Look at anything in user, Temp, or ProgramData paths.
4. **Overview → Deleted Files.** Tools removed after use.
5. **Index Search** for indicators from live triage and Vol (IPs, filenames, PIDs' image names).
6. **Explore** the §7B paths for persistence and logs.

File List controls:

- **Filter** dropdown (toolbar). Start with **Unfiltered**, then try built-ins like **Actual Files** to drop slack and free space. Make your own via **Filter Manager** (e.g. Modified > date AND Category = Executable).
- **Column settings** (right-click header). Add Created / Modified / Accessed, Path, MD5, Bad Extension.
- **KFF Ignorable** filter hides known-good OS files. It shrinks the list dramatically.

On any hit:

1. Bottom viewer: **Natural / Text / Hex / Filtered**
2. Right-click → **Create Bookmark** (name it after the finding, add a comment)
3. Right-click → **Export** to hash elsewhere or feed to Vol / strings
4. Registry hives: right-click → **View in Registry Viewer** (separate tool) for Run keys, UserAssist, ShellBags

5. Pivot on the timestamp: sort the Explore list for that directory by Modified

Report: `File → Report` → include **Bookmarks** only, plus hashes and file paths → HTML or PDF.

---

8. Autopsy

8A. Setup

New case:

1. `Case → New Case` → name `CASE_<host>` → base directory on output drive → Next → examiner info → Finish
2. **Add Data Source** wizard opens. Host: Generate new host name based on data source name → Next.
3. Data source type:
   - **Disk Image or VM File** — E01, raw/dd, VMDK, VHD (most common)
   - **Local Disk** — a mounted or attached drive (FTK-mounted image works here)
   - **Logical Files** — a folder of exported artifacts
4. Pick the file → time zone = **the target's** time zone → Next. To display times in UTC: `Tools → Options → View` → time zone → **GMT/UTC**.
5. **Ingest modules.** Fast set first, slow ones later.

Run first (fast)	Run later if time (slow)
Recent Activity	Keyword Search (full index)
File Type Identification	PhotoRec Carver
Hash Lookup (+ NSRL if loaded)	Plaso (full timeline)
Extension Mismatch Detector	Embedded File Extractor
Interesting Files Identifier	Encryption Detection
Data Source Integrity	YARA Analyzer (if present)

6. Finish. Ingest runs in the background. Watch the progress bar bottom-right. Results appear as they finish, so start browsing immediately.

Add a quick keyword list: `Tools → Options → Keyword Search → New List` → add terms (attacker IP, filename, `base64`, `eval(`) → run **Keyword Search** ingest against the data source.

Memory images: if Add Data Source offers a memory/Volatility option, it runs embedded Vol2 plugins on Windows images only. Otherwise do memory in Vol3 (§4–§5) and bring dumped files back as **Logical Files**.

Run ingest again later: right-click data source → **Run Ingest Modules**.

8B. Hunt Navigation

Tree layout (left). Work it top-down. Autopsy 4.19+ labels these **Data Artifacts / Analysis Results**; older builds put them all under **Results → Extracted Content**.

Node	Use it for
Data Sources	Raw file system browse (same as FTK tree)
File Views → File Types → By Extension / MIME	All `executable` , `.ps1` , `.bat` , `.php` , `.sh` at once
File Views → Deleted Files	Recently deleted tooling
File Views → File Size	Large archives (staging / exfil)
Data Artifacts → Run Programs	Prefetch-based execution
Data Artifacts → Recent Documents · **Shell Bags**	What a user opened / browsed
Data Artifacts → Installed Programs · **USB Device Attached**	New software, removable media
Data Artifacts → Web History / Downloads	How a payload arrived
Data Artifacts → Operating System User Account	Accounts, last logon
Analysis Results → Extension Mismatch	`.jpg` that's really a PE
Analysis Results → Interesting Items · **Hashset Hits**	Rule and hash matches
Analysis Results → Keyword Hits	Your keyword lists
Tags	Your bookmarked evidence (feeds the report)

Timeline: `Tools → Timeline` . Set the zoom to the window around a known-bad time. Filter by event type (file modified, program run, web). Look for clusters around the suspicious timestamp. This is the fastest way to "what happened right before and after."

Workflow on any hit:

1. Select the item → bottom pane: **Hex / Text / File Metadata / Analysis Results**
2. Right-click → **Add File Tag → Notable Item** (comment why)
3. Right-click → **View File in Directory** to see its neighbors
4. Right-click → **Extract File(s)** to hash or feed to other tools
5. Note the timestamp → jump to Timeline

Report: `Generate Report → HTML` (or Excel). Choose **Tagged results only** for a clean findings export.

9. Findings Log (keep while you work)

Keep it in **Hunt Notes** (`tools/hunt-notes.html`): paste the time as shown, pick the source zone, it stores UTC and builds the timeline, IOC list and coverage grid. Its Output tab prints this same format. Paper fallback, one line per observation:

```
TIME (UTC)           | HOST   | SOURCE          | Q  | CONF     | OBSERVATION
| EVIDENCE / CMD                | NEXT
---------------------+--------+-----------------+----+----------+-----------------------------------
-----+-------------------------------+-------------
2026-10-21 14:05:12Z | WIN-01 | live:Win32_Proc | Q1 | Possible | powershell -enc child of winword
(4412)  | Get-CimInstance Win32_Process | netscan 4412
2026-10-21 14:05:40Z | WIN-01 | vol3:netscan    | Q2 | Probable | 4412 -> 203.0.113.7:443 ESTABLISHED
| windows.netscan               | dump 4412
2026-10-21 03:58:02Z | LNX-01 | access.log      | Q4 | Possible | POST /uploads/x.php from
198.51.100.4    | access.log L8812              | find x.php
```

Report-out per finding:

```
FINDING:     <one sentence>
HOST:        <name / IP>
WHEN:        <first seen UTC> → <last seen UTC>
EVIDENCE:    <artifact + tool + location>
VALIDATION:  <second, independent source that confirms it>
ATT&CK:      <tactic / technique ID>
MITIGATION:  <contain / remove / harden>
```

10. Detection — KQL Host Hunts

Run in Kibana Discover, as saved searches, or as the basis for detection rules. KQL unless marked ES|QL.

```
# Office / email spawning a shell  (T1566 / T1059)
process.parent.name:("winword.exe" or "excel.exe" or "powerpnt.exe" or "outlook.exe")
  and process.name:("cmd.exe" or "powershell.exe" or "wscript.exe" or "cscript.exe" or "mshta.exe")

# Encoded / download-cradle PowerShell  (T1059.001 / T1105)
process.name:"powershell.exe" and process.command_line:
  (*-enc* or *-EncodedCommand* or *FromBase64String* or *IEX* or *Invoke-Expression*
   or *DownloadString* or *Net.WebClient* or *-nop* or *-w hidden*)

# LOLBins with suspicious args  (T1218)
process.name:("rundll32.exe" or "regsvr32.exe" or "mshta.exe" or "certutil.exe"
   or "bitsadmin.exe" or "msiexec.exe")
  and process.command_line:(*http* or *\\temp\\* or *-urlcache* or *javascript:* or *.dll,*)

# LSASS access — credential dumping  (T1003.001)  Sysmon 10 / Defend
event.code:"10" and winlog.event_data.TargetImage:*lsass.exe*
  and not winlog.event_data.SourceImage:("*\\MsMpEng.exe" or "*\\wininit.exe")

# New service / scheduled task persistence  (T1543.003 / T1053.005)
event.code:("7045" or "4697" or "4698" or "4702")

# Lateral movement via explicit creds  (T1021 / T1078)   then pivot one user.name -> many host.name
event.code:"4648"

# Failed-logon spray / bruteforce  (T1110)   aggregate by source.ip / user.name, threshold on count
event.code:"4625"

# Linux: suspicious execution from web/temp dirs  (T1505.003 webshell / T1059.004)
host.os.type:"linux" and process.executable:("/tmp/*" or "/dev/shm/*" or "/var/www/*")
  and process.parent.name:("apache2" or "httpd" or "nginx" or "php-fpm")
```

11. Detection — KQL / ES|QL Network Hunts

```
# Beaconing candidates — fixed-interval small conns to one external dst  (T1071)   ES|QL:
FROM logs-zeek.*
| WHERE network.direction == "outbound"
| STATS conns = COUNT(*), bytes = SUM(network.bytes), uniq_dur = COUNT_DISTINCT(zeek.conn.duration)
    BY source.ip, destination.ip, destination.port
| WHERE conns > 20 AND bytes < 500000
| SORT conns DESC

# Rare external destinations (first-seen)
destination.ip:* and not destination.ip:(<internal ranges>) and not destination.ip:(<baseline
externals>)

# DNS tunneling / DGA markers  (T1071.004 / T1568)
zeek.dns.query:* and (dns_qlen > 50 or dns_entropy > 4)
zeek.dns.qtype_name:"TXT"
dns.response_code:"NXDOMAIN"

# Self-signed / odd TLS  (T1573 / C2 over TLS)
tls.server.x509.issuer.distinguished_name : tls.server.x509.subject.distinguished_name
  or tls.client.ja3:(<known-bad JA3>)
  or (tls.client.server_name:* and not destination.ip:<expected for that SNI>)

# Long-lived external sessions
zeek.conn.duration > 3600 and network.direction:"outbound"
```

12. Detection — Suricata

Rule anatomy:

```
action proto src_ip src_port -> dst_ip dst_port (msg:"..."; flow:...; content:"..."; \
   pcre:"..."; classtype:...; sid:1000001; rev:1; metadata:...;)
```

Custom examples (defensive, terrain-specific):

```
# Outbound cleartext to a known-bad / unexpected external on a monitored egress
alert tcp $HOME_NET any -> $EXTERNAL_NET any (msg:"CUSTOM Egress to flagged host"; \
   flow:established,to_server; ip_dst <flagged-ip>; classtype:trojan-activity; sid:9000001; rev:1;)

# Self-signed TLS to external (metadata)
alert tls $HOME_NET any -> $EXTERNAL_NET any (msg:"CUSTOM Self-signed cert outbound"; \
   tls.cert_issuer; content:!"CN="; classtype:protocol-command-decode; sid:9000002; rev:1;)

# SMB lateral indicator (admin share from a workstation subnet)
alert smb $WKSTN_NET any -> $SERVER_NET 445 (msg:"CUSTOM Wkstn->Server admin share"; \
   flow:to_server; classtype:policy-violation; sid:9000004; rev:1;)
```

Baseline suppression (threshold.config) — every suppress cites its BL-ID:

```
# Suppress a sig entirely from the range NAT/jump source  (BL-003 [RANGE])
suppress gen_id 1, sig_id 2000419, track by_src, ip <range-nat-ip>

# Rate-limit a noisy-but-real sig to one alert per 5 min per host
threshold gen_id 1, sig_id 9000003, type threshold, track by_src, count 1, seconds 300
```

13. Detection — Zeek

Log	What it gives	Hunt use
`conn.log`	every flow: duration, bytes, state, history	beaconing, long conns, top talkers, exfil ratios
`dns.log`	queries, responses, rcodes, types	DGA, tunneling, NXDOMAIN spikes, TXT volume
`http.log`	method, host, URI, UA, status	webshell URIs, rare UAs, suspicious downloads
`ssl.log + x509.log`	SNI, versions, cert subject/issuer, JA3/JA3S	self-signed, rare JA3, SNI/IP mismatch
`files.log`	files seen, MIME, hashes	malware delivery, carving
`notice.log / weird.log`	Zeek detections / protocol anomalies	pre-built notices, tunneling, evasion

Notice framework — turn a hunt into a standing detection:

```
# long-conns.zeek  — notice on outbound sessions over 1h
@load base/frameworks/notice
redef enum Notice::Type += { Long_Outbound_Conn };
event connection_state_remove(c: connection) {
    if ( c$duration > 1hr && Site::is_local_addr(c$id$orig_h)
         && ! Site::is_local_addr(c$id$resp_h) )
        NOTICE([$note=Long_Outbound_Conn, $conn=c,
                $msg=fmt("Long outbound conn %s -> %s (%s)", c$id$orig_h, c$id$resp_h, c$duration)]);
}
```

Baseline exclusion — suppress the ism in the notice policy, keep the log, cite the register ID:

```
hook Notice::policy(n: Notice::Info) {
    if ( n$note == Long_Outbound_Conn && n$conn$id$resp_h == <backup-server> )
        break;   # BL-014 [BOTH] confirmed: nightly backup, not a finding
}
```

14. Detection — Elastic Defend

Mechanism	What it does	Use for
Event filter	Stops matching events being written/ingested at the agent	High-volume known-good (range orchestration, traffic-generator noise). Cheapest, earliest.
Rule exception	Stops one rule alerting on a match, event still ingested	Known-good that trips one rule but the raw event still has hunt value.
Trusted application	Excludes a binary/path/signer from protections	A sanctioned tool the malware/behavior engine flags.

```
# Event filter  (BL-007 [RANGE] traffic generator)
event.category : "process" and process.parent.name : "<traffic-gen>.exe"

# Rule exception on "Certutil URL download"  (BL-022 [CLIENT] confirmed admin host)
host.name : "ADM-TOOLS-01" and user.name : "svc-patch"
```